

Guardrails Challenge

Building a Modern Guardrails System for Conversational AI

Overview

We're excited to see how you approach building safety mechanisms for LLM applications! This task gives you the freedom to explore modern guardrails techniques while demonstrating your understanding of production AI challenges.

The core challenge: Design and implement a guardrails system that makes a conversational AI safer, more reliable, or more policy-compliant. How you approach this is entirely up to you - we're interested in seeing your thinking process, technical choices, and creativity.

The Challenge

Imagine you're building a conversational AI for a domain where mistakes matter - maybe healthcare, finance, education, customer support, or anything else you find interesting. Your system needs mechanisms to prevent or mitigate failures.

What kinds of failures might you consider?

Think about scenarios like:

- Users trying to manipulate the system (prompt injections, jailbreaks)
- The model generating inaccurate or hallucinated information
- Violating domain-specific policies or ethical guidelines
- Producing outputs in the wrong format or structure
- Drifting off-topic during longer conversations
- Exposing sensitive information inappropriately

Pick 3-5 failure modes you find most interesting or relevant to your chosen domain.

Your Approach - Many Paths Forward

There's no single "right" way to solve this! Here are some directions you might explore:

Input-Side Guardrails

Could involve detecting problematic inputs, classifying intent, analyzing conversation history, checking for injection attacks, or validating requests against policies. You might use ML classifiers, LLM-based judges, similarity searches, or rule-based systems.

Output-Side Guardrails

Could include validating responses for accuracy, checking format compliance, detecting hallucinations, implementing self-critique loops, structured output enforcement, or cross-checking with knowledge bases. Techniques might range from Pydantic validators to multi-model verification.

Frameworks & Tools - Use What Excites You!

Feel free to explore any tools that interest you:

- Modern guardrails frameworks (Guardrails AI, NeMo Guardrails, Llama Guard)
- Orchestration tools (LangChain, LangGraph, DSPy, Haystack)
- Structured output libraries (Instructor, Outlines, Marvin, JSONFormer)
- Or build something custom from scratch if you prefer!

You could even mix approaches - maybe a lightweight custom solution with strategic use of third-party APIs where it makes sense.

What We'd Love to See

1. Your Design Thinking (Document in your preferred format)

Share your reasoning:

- What domain did you choose and why?
- Which failure modes are you addressing?
- What's your overall architecture approach?

- What trade-offs did you consider? (speed vs safety, simplicity vs coverage, cost vs reliability)
- How will you know if it's working?

A diagram or flowchart could be helpful, but whatever communicates your ideas clearly works!

2. Implementation (Structure it however makes sense to you)

Show us working code that demonstrates your guardrails. This could be:

- A notebook walking through your approach
- A clean Python module with good abstractions
- A framework-based implementation showcasing integration
- An API wrapper with middleware logic
- Whatever format best showcases your solution!

We care more about **clear thinking and solid execution** than specific structure.

3. Evidence It Works

Help us understand the impact of your guardrails:

- Some test cases showing before/after behavior
- Ideally including some challenging scenarios (adversarial inputs, edge cases)
- Metrics that matter to you (accuracy, latency, cost, false positives)
- Honest discussion of where it struggles

Quality over quantity - a few well-analyzed examples beat many superficial ones.

4. Reflection

Share your thoughts on:

- What worked well in your approach?
- What limitations or gaps remain?
- What would you add/change with more time or resources?
- Any interesting surprises during implementation?

What Makes a Submission Stand Out?

We're looking for evidence of:

- **Thoughtful problem-solving** - showing why you made specific choices
- **Technical competence** - clean code, proper use of tools, understanding of concepts
- **Realistic thinking** - acknowledging trade-offs and limitations
- **Creative approaches** - maybe you found an interesting angle we haven't seen!
- **Practical mindset** - considerations for how this might work in production

What We're NOT Looking For

Don't stress about:

- Using specific frameworks or tools (use what you're comfortable with!)
- Perfect security (impossible anyway!)
- Massive test suites (focused quality beats comprehensive quantity)
- Enterprise-grade production code (though good practices are appreciated)
- Covering every possible failure mode (pick a focused scope)

Time Guidance

Most people spend **4-8 hours** on this, but adjust based on your schedule:

- Lighter approach (4-5 hours): Pick 2-3 guardrails, implement core functionality, basic testing
- Standard approach (6-7 hours): Multiple guardrail types, good test coverage, solid documentation
- Extended approach (8+ hours): Add bonus elements that interest you!

Quality and clear thinking matter more than hours spent.

Some Ideas to Spark Creativity

If you're looking for inspiration (totally optional!):

- Build a red-teaming tool that tests your own guardrails

- Implement confidence scoring that adapts safety strictness
- Create visualization of guardrail trigger patterns
- Design a fallback chain when primary guardrails activate
- Experiment with having the model explain why guardrails triggered
- Build a simple A/B testing framework for different guardrail configs

Getting Started

Here's one possible approach (but feel free to ignore this!):

1. **Explore** - Pick a domain, brainstorm 3-5 risks you find interesting
2. **Prototype** - Get one guardrail working end-to-end
3. **Expand** - Add 1-2 more layers or techniques
4. **Test** - Try to break your system, document what happens
5. **Reflect** - Write up your thinking and findings

Submission & Delivery

Preferred Method: GitHub Repository

We'd love to see your work as a **GitHub project**! This allows us to:

- Review your code structure and organization
- See your commit history and development process
- Check out your documentation style
- Easily run and test your implementation

Suggested Repository Structure

```
your-guardrails-project/
├── README.md           # Project overview and setup instructions
├── DESIGN.md           # Your design thinking and architecture
├── requirements.txt     # Dependencies
├── src/
│   ├── guardrails/     # Your guardrail implementations
│   └── chatbot.py       # Main chatbot logic
```

```
|   └── utils.py          # Helper functions
|   └── tests/
|       ├── test_cases.py    # Your test scenarios
|       └── test_results.md  # Test results and analysis
|   └── examples/
|       ├── demo.ipynb      # Optional: Jupyter notebook demo
|       └── example_runs.md  # Example interactions
|   └── docs/
|       └── evaluation.md    # Your reflection and evaluation
```

Feel free to adapt this structure to what makes sense for your approach!

What to Include in Your Repository

Essential:

- Clear README with setup instructions
- Your implementation code
- Documentation of your design decisions
- Test cases and results
- Reflection on your approach

Nice to have:

- Requirements file or poetry/pipenv config
- Demo notebook or script showing usage
- Diagrams or visualizations
- CI/CD setup (if you're feeling ambitious!)

Alternative Delivery Methods

If GitHub isn't feasible for some reason, you can also submit:

- A ZIP file with the same structure
- A Google Colab notebook (for simpler implementations)
- GitLab, Bitbucket, or other git hosting

Just make sure we can access it easily!

Before You Submit

Quick checklist:

- ☐ Can someone else clone and run your project?
- ☐ Are dependencies clearly listed?
- ☐ Is there a clear entry point (how to run it)?
- ☐ Have you documented your key decisions?
- ☐ Are API keys and secrets handled safely? (.env file, clear instructions)
- ☐ Does your README explain what the project does?

Questions?

This task is intentionally open-ended. If you're unsure about scope, just make reasonable choices and document your reasoning. We're more interested in **how you think** than whether you guessed what we wanted.

We can't wait to see what you build! Show us your approach to making AI systems safer and more reliable. Good luck! 🚀

Final note: There's no perfect solution here. Real-world guardrails are always a work in progress. We value honest discussion of limitations over claims of bulletproof systems. Have fun with it!

Need Help?

If you have questions about the task or run into technical issues, feel free to reach out. We're here to help you succeed!